



Python 2: Visualization and Analysis

We Unlock Career Potential

New York | Chicago | San Francisco | Charlotte | New Delhi | London | Hong Kong

© 1999-2026 *Training The Street LLC*

All rights reserved

www.trainingthestreet.com

© 1999-2026 Training The Street LLC
All rights reserved.

Training The Street LLC (“TTS”) owns all rights, including copyright, in this publication. This publication may not be reproduced or redistributed, in whole or in part, in any format, or by any means without TTS’s prior written consent. No TTS seminar, workshop, or other instructional activity may be recorded or transmitted in any format or by any means without TTS’s prior written consent.

We Unlock Career Potential

www.trainingthestreet.com

About Training The Street

Founded in 1999, [Training The Street](#) is a leading financial learning services company, successfully delivering targeted and customized training courses to corporate and educational clients.

Our client list includes over 250 institutions representing the world's leading banks, accounting firms, private equity firms, consulting firms, Fortune 100 companies, and the most respected universities globally.

Several ways to personally unlock your career potential:



Public Courses

Our turnkey finance training courses make new, lateral and off-cycle hires 'desk-ready' productive members of your team in no time.



Self-Study Courses

Our video based self-study courses use the same content and the same instructors to put you in control so you can learn at your own pace.



TTS Turbo Macros

Boost your productivity in Excel and enhance your workflow making you faster and more efficient.
Accelerate your Excel skills

Core Training Capabilities

Fundamental Content	Sector Capabilities Training
Accounting	Commercial Real Estate
Corporate Valuation	FIG Analysis (Banks & Insurance)
Financial Modeling	Financial & Corporate Restructuring
M&A Modeling	Oil & Gas
LBO Modeling	Project Finance
Capital Markets (DCM / ECM Origination)	Private Equity Investing & Analysis
Financial Products / Global Markets (S&T)	Infrastructure
Portfolio / Investments Analysis	Investment and Wealth Management
Excel Best Practices	Software (SaaS) Analysis
Specialist / Functional Capabilities	Venture Capital
Corporate Credit Analysis	Renewables
Cash Mgt & Treasury Services	Data Science
ESG	AI & LLMs
Investment Banking Overview	Applied Excel
Investment Authorities	Google Sheets
Private Co. Analysis	Microsoft Copilot
PowerPoint	Python
Data Storytelling	Power BI
FP&A	SQL
	Tableau
	VBA

Table of Contents

- Using *matplotlib*
 - Summary & Scatter / Time Series Plots
 - Multiple Graphs
 - Histograms and Box Plots
 - Bubble Charts
- Using *statsmodels*
 - Linear Regression
 - Time Series
- Using *SciPy* – Optimization
 - Portfolio Optimization
- Using *scikit-learn (sklearn)*
 - K-Means Clustering
 - Principal Component Analysis

Using *matplotlib*

Using *matplotlib*

- Matplotlib is a 2D visualization package
 - Makes graphs similar to those in MATLAB
 - Useful for time-series and cross-sectional data
- Documentation
 - https://matplotlib.org/api/_as_gen/matplotlib.pyplot.html#module-matplotlib.pyplot
- It is common practice to import the package as plt
 - Predominantly will be importing pyplot

```
# %% Import Packages
```

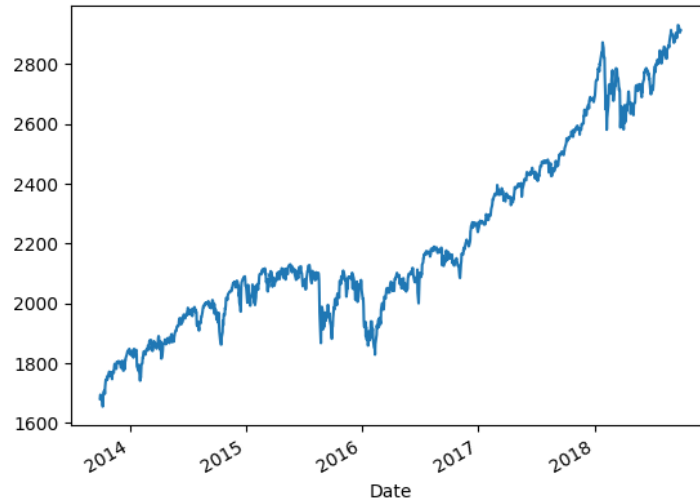
```
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import pandas as pd
import numpy as np
import os
```

Using *matplotlib*

Summary & Scatter / Time Series Plots

Using *matplotlib* – Scatter Plots

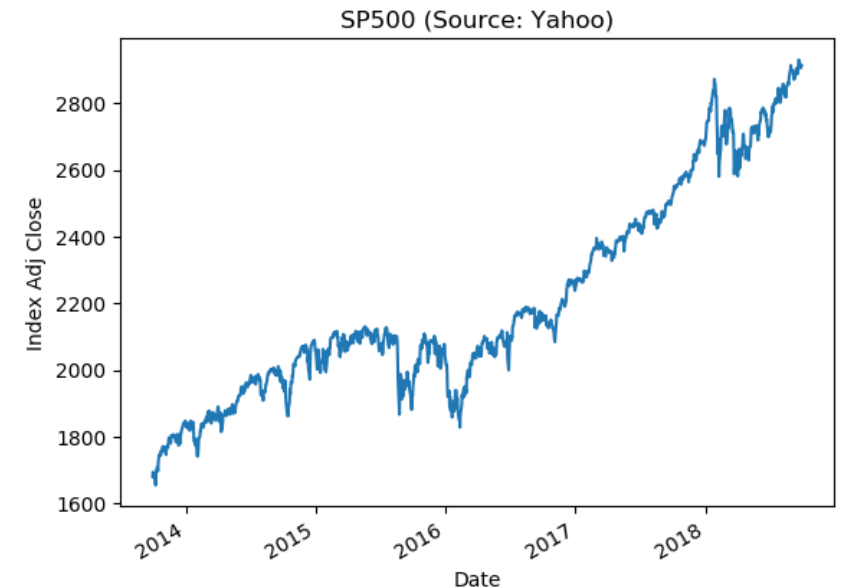
- Scatter plots are used to plot relationship between two variables
 - Including time series
 - Remember X (horizontal axis), Y (vertical axis)
- Leveraging integration and built in functionality to quickly create a plot
 - Can call `.plot()` from a *pandas* data frame
 - `plt.show()` is the command to display the plot



Using *matplotlib* – Scatter Plots

- Adding axis labels is good practice
 - Since the package understands the series date index, the axis is labeled automatically
 - All formatting is done before the `plt.show()` command
- Axis labels
 - `.xlabel()` & `.ylabel()` use strings for inputs
 - `.title()` also uses a string as an input

```
>>> plt.ylabel("Index Adj Close")  
>>> plt.title("SP500 (Source: Yahoo)")
```



Using *matplotlib* – Scatter Plots

- Annotations and markers can be added to the graph
 - `.annotate(s, xy, xytext, arrowprops)`
 - **s**: The text to display
 - **Xy**: The point to annotate
 - **Xytext**: the position to place text
 - **Arrowprops**: The properties used to draw the arrow patch
 - First, the location is needed for the point that will be highlighted, the xy coordinates
 - `.max()` or `.min()` will retrieve the value (y-axis)
 - `.idxmax()` or `.idxmin()` will retrieve the index (x-axis)

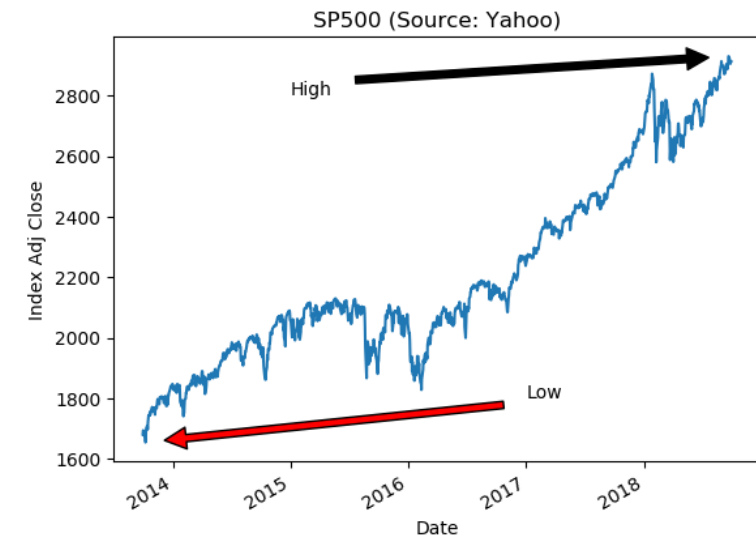
```
>>> spmax = sp500['Adj Close'].max()
>>> spmaxid = sp500['Adj Close'].idxmax()

>>> spmin = sp500['Adj Close'].min()
>>> spminid = sp500['Adj Close'].idxmin()

>>> plt.annotate('High', xy=(spmaxid, spmax),
                 xytext=("2015-01-01", 2800),
                 arrowprops=dict(facecolor="black", shrink=0.05))

>>> plt.annotate('Low', xy=(spminid, spmin),
                 xytext=("2017-01-01", 1800),
                 arrowprops=dict(facecolor="red", shrink=0.05))

>>> plt.show()
```



Using *matplotlib*

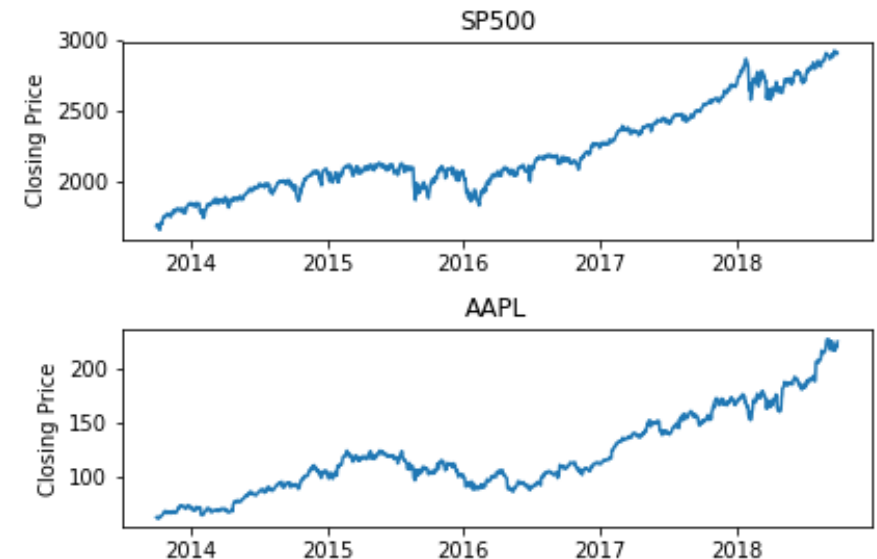
Multiple Graphs

Using *matplotlib* – Multiple Graphs

- `.subplot(nrows, ncols, index)`
 - Creates an object that is the figure which holds the individual plots
 - When called again before `.show()` is called, it will index to the next plot in the figure
 - Syntax:
 - **Nrows**: specify the number of rows for the subplot grid
 - **Ncols**: specify the number of columns for the subplot grid
 - **Index**: specify the index, starting at 1 in the top left corner, moving right then down of which position currently being provided plot instructions

```
#Create a subplot 2 rows, 1 column.  
#Display S&P500 and Apple Data  
>>>plt.subplot(2,1,1)  
>>> plt.plot(sp500.index, sp500['Adj Close'])  
>>> plt.title('SP500')  
>>> plt.ylabel('Closing Price')
```

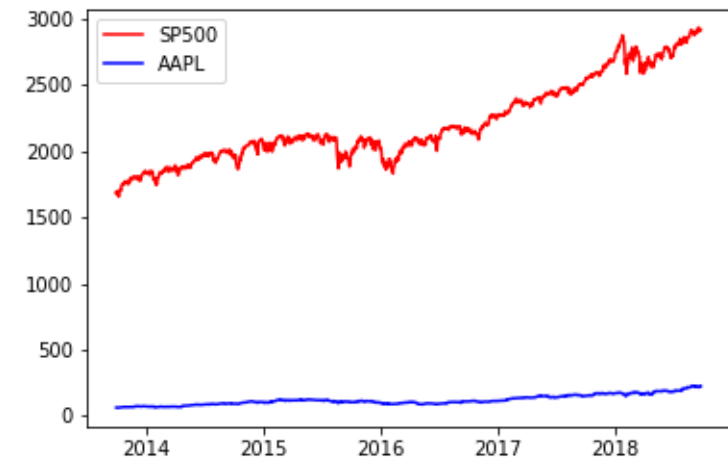
```
#Index to next plot for Apple  
>>> plt.subplot(2,1,2)  
>>> plt.plot(aapl.index, aapl['Adj Close'])  
>>> plt.title('AAPL')  
>>> plt.ylabel('Closing Price')  
>>> plt.savefig('figure1.png')  
>>> plt.show()
```



Using *matplotlib* – Multiple Graphs

- Plotting Multiple Lines on the same graph
 - Graphs can be added to the same plot before `.show()` is called
 - Simply specify another plot command
- Notice the behaviour
 - The axis is scaled based on the stock market magnitude, which is suitable for the Index, but not meaningful for the Apple Stock

```
>>> plt.plot(sp500.index,  
             sp500['Adj Close'], 'red')  
>>> plt.plot(aapl.index,  
             aapl['Adj Close'], 'blue')  
  
#pass the names of the stocks in the order created above  
>>> plt.legend(['SP500', 'AAPL'])  
>>> plt.show()
```



Using *matplotlib* – Multiple Graphs

- `.subplots(nrows=1, ncols=1)`
 - Returns a figure and axes object
 - The figure is a container for the plot elements
 - The axes contain the individual figure elements
 - Syntax:
 - Nrows: defaults to 1 if not specified (single plot)
 - Ncols: defaults to 1 if not specified (single plot)
- `ax.twinx()` creates a new Axes object with the x-axis inherited from the original axes
- `fig.tight_layout()` reduces the padding between subplots

```
#Create a subplot object for figure and axis
>>> fig, ax1 = plt.subplots()
>>> ax1.plot(sp500.index, sp500['Adj Close'], 'black', alpha=0.5)
>>> ax1.set_ylabel('Index Close', color='black')
#Change the appearance of ticks, tick labels, and gridlines.
>>> ax1.tick_params('y', colors='black')
>>> ax2 = ax1.twinx()
>>> ax2.plot(aapl.index, aapl['Adj Close'], 'blue')
>>> ax2.set_ylabel('AAPL Close ($)', color='blue')
>>> ax2.tick_params('y', colors='blue')
>>> fig.tight_layout()
>>> fig.show()
```

Using *matplotlib* – Multiple Graphs

- Now the scaling is correct, and a relationship can be observed
- Arguments are passed to several of the axis methods to format the graph
 - **Alpha**: takes a float between 0 and 1 to control the transparency of the line (1 is solid)
 - **Color**: specifies the color (line, text)
 - **Marker**: string or number which specifies a variety of symbols



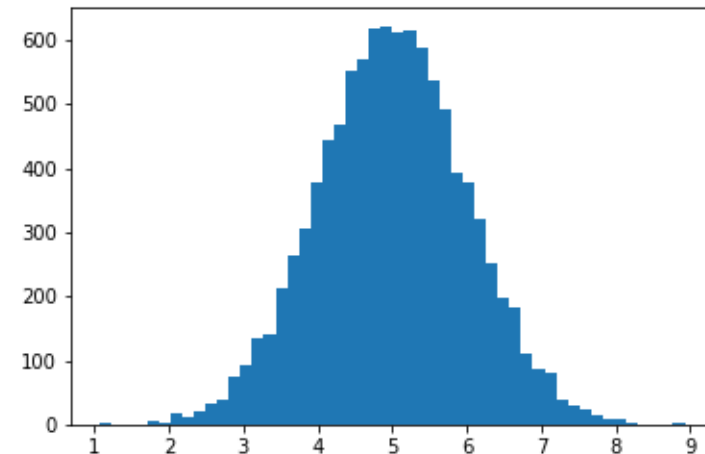
Using *matplotlib*

Histograms and Box Plots

Using *matplotlib* – Hist and Box Plots

- Histograms
 - Useful in displaying counts of binned data
 - Can be used to visualize a distribution
 - `.hist(x, bins=None, density=None)`
 - **X**: is the data to display in the histogram
 - **Bins**: the number of “buckets” used to count the frequency
 - **Density**: if set to True, will normalize the histogram so the area equals one (not the number of observations)

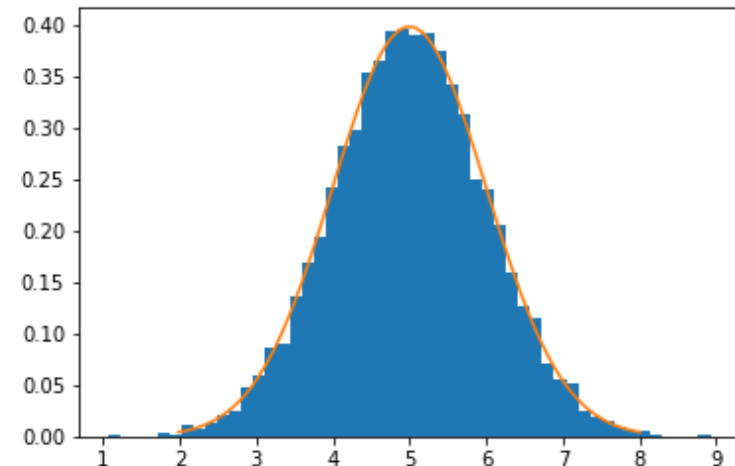
```
#Create a series of random draws
>>> np.random.seed(42)
>>> ndraws = np.random.normal(
    loc=5, scale=1, size=10000)
#Create a histogram
>>> plt.hist(ndraws, bins=50)
>>> plt.show()
```



Using *matplotlib* – Hist and Box Plots

- To add a line to the histogram
 - Going to overlay the true (analytical) normal distribution
 - After the histogram is created using `plt.hist()` specify the line using `plt.plot()`
 - Note that in this case density is set to `True` for the histogram which will normalize the histogram and match the magnitude of the line
 - An alternative option is to add an additional axis

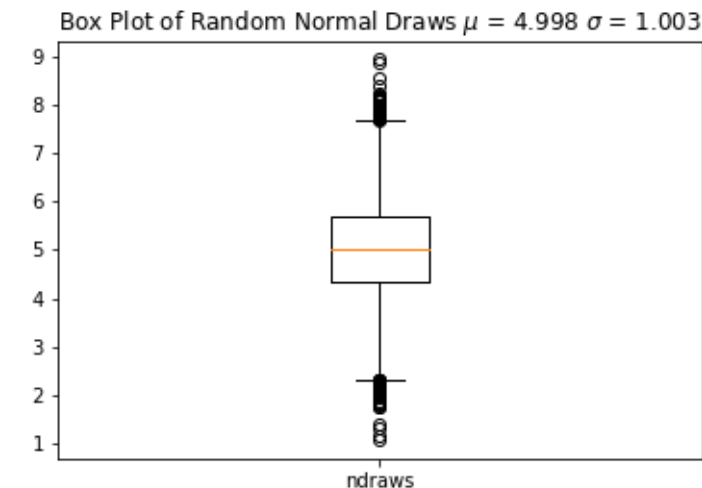
```
>>> x_numbers = np.linspace(
    ndraws.mean() - 3*ndraws.std(),
    ndraws.mean() + 3*ndraws.std(),
    100)
>>> plt.hist(ndraws, bins=50,
    density = True)
>>> plt.plot(x_numbers, norm.pdf(x_numbers,
    ndraws.mean(), ndraws.std()))
>>> plt.savefig('figure1.png')
>>> plt.show()
```



Using *matplotlib* – Hist and Box Plots

- A Box Plot helps visualize the distribution of data
 - Highlights the lower and upper quartile, with a box and whisker respectively
 - Marks the outliers, box is centered at the median (middle)
 - Useful in comparing distributions across variables
 - `.boxplot(x, notch=False, bootstrap=None)`
 - **X**: data for the box plot
 - **Notch**: Set to true to display confidence intervals
 - **Bootstrap**: specify an integer value if the desire is to calculate bootstrapped confidence intervals instead of asymptotic intervals

```
>>> nmu = ndraws.mean()
>>> nstd = ndraws.std()
>>> fig, ax = plt.subplots()
>>> ax.boxplot(ndraws)
>>> ax.set_xticklabels(['ndraws'])
>>> ax.set_title("Box Plot of Random Normal Draws  $\mu = %.3f$   $\sigma = %.3f$ " % (nmu, nstd))
>>> fig.savefig('figure1.png')
>>> fig.show()
```



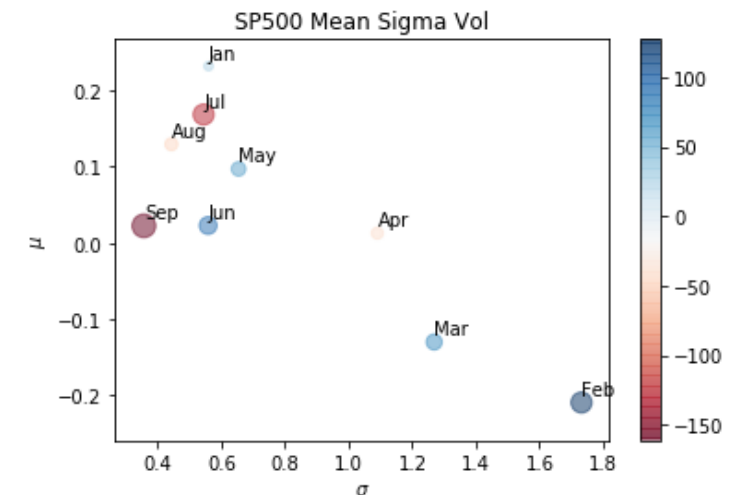
Using *matplotlib*

Bubble Charts

Using *matplotlib* – Bubble Charts

- Bubble Charts
 - Can be used to represent a 3D scatter plot in a 2D plane making it easier to comprehend
 - `.scatter(x, y, s=None, c=None, cmap=None, alpha=None)`
 - **X, Y**: location data similar to a scatter or line graph
 - **S**: data to determine the size of the marker. Depending on the scale of the axis may need to scale the size data.
 - **Cmap**: will color code the data based on the gradient selected
 - **C**: data which drives the color selection

```
plt.scatter(bubble_data.sigma,  
            bubble_data.mu,  
            s=np.abs(bubble_data.normvol) * 1000,  
            c=bubble_data.normvol,  
            cmap=plt.cm.get_cmap('RdBu'),  
            alpha=0.5)
```



Using *matplotlib* – Bubble Charts

- Annotation
 - Text can be added to any graph using `plt.annotate()`
 - Important for bubble charts to know what each point references
 - Can be automated using a for loop
 - Iterate over the data (they are the x, y coordinates)
 - A list can be used to reference, or iterate over the text
- The `zip()` function
 - Takes two or more iterables, and creates tuples based on the index entries

```
>>> import calendar

>>> for x, y, mon in zip(bubble_data.sigma, bubble_data.mu, bubble_data.index):
    plt.annotate(calendar.month_abbr[mon[0]], xy = (x,y+0.01))

# Example of how zip() functions

>>> for x, y, z in zip(range(0,3), range(3,6), range(6,9)):
...     print('Value of X: {}, Value of Y: {}, Value of Z: {}'.format(x,y,z))
...
Value of X: 0, Value of Y: 3, Value of Z: 6
Value of X: 1, Value of Y: 4, Value of Z: 7
Value of X: 2, Value of Y: 5, Value of Z: 8
```

Using *statsmodels*

Using *statsmodels*

- Statsmodels is a python package that provides functions to perform linear and time series analysis
 - Similar to R for linear regression and time series
- This lesson will focus on
 - Linear Regression (CAPM)
 - Time Series (ARMA)
 - Visualizing Analysis

```
# %% Packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import statsmodels.api as sm
import statsmodels.tsa as tsa
```

Using *statsmodels*

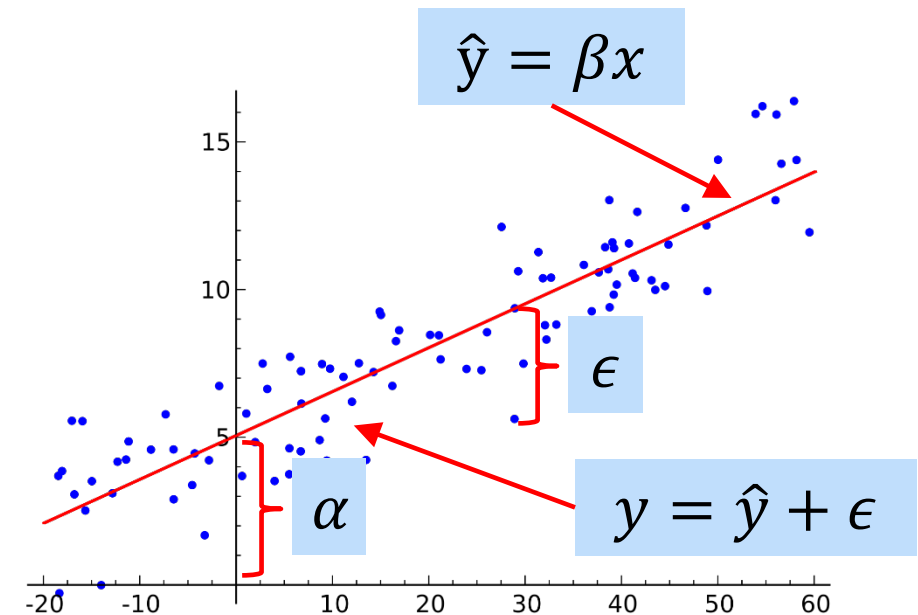
- Organizing data
 - It is best to organize the data into a single DataFrame
 - Verify the index is aligned, then handle missing data accordingly
- After the data is cleaned, copy the DataFrame into an array and if necessary, scale the data and check for outliers
 - For numerical stability, try to keep numbers close to one
 - There are several techniques to handle outliers
 - Normalization: Transform the data to be zero mean and unit variance
 - Winsorizing: replace the outliers with the value contained at 90th, 95th or 99th percentile
 - Log Transform: take the log of the data

```
>>> sp500 = pd.read_csv('StockData/SP500.csv', header=0, index_col=0, parse_dates=True)
>>> aapl = pd.read_csv('StockData/aapl.csv', header=0, index_col=0, parse_dates=True)
# Need to cast the series as a DataFrame to merge
>>> data = pd.DataFrame(sp500['Adj Close']).merge(aapl['Adj Close'], left_index=True, right_index=True)
>>> data.columns = ['sp500', 'aapl']
>>> data[['sp500_rtns', 'aapl_rtns']] = np.log(data[['sp500', 'aapl']]).diff()
>>> data[['sp500_rtns', 'aapl_rtns']] *= 100
>>> data.dropna(inplace=True)
```

Linear Regression with *statsmodels*

Linear Regression

- The simplest form of linear regression is finding the line of “best fit” between two variables, the dependent and independent variable
 - Traditionally the dependent variable is called endogenous and labelled as y
 - The independent variables are called exogenous and labelled as x
 - The error term is what is left over, not explained by the exogenous variable, and is labelled as ϵ
 - The general form equation is $y = \alpha + \beta x + \epsilon$
 - The slope, beta, is the sensitivity of the dependent variable to the independent variable
 - Typically thought of as a unit change in x results in a beta change of y
 - Best fit is found by minimizing the least squares objective function $\min_{\beta} \sum (y_i - \beta x_i)^2$



Linear Regression

- This simple two variable model can be expanded into a multivariate model
 - The equation doesn't change, $y = X\beta + \epsilon$, but now X is a matrix with multiple variables used to explain y including a column of ones for the intercept
- The following assumptions are made for the model
 - **The model is correctly specified:** a linear model is specified, thus the relationship in reality should be linear (not exponential, square etc.)
 - **Exogeneity:** $E[\epsilon|X] = 0$ the conditional mean of the error term is zero based on the exogenous variables. Be aware for this assumption that the objective function will make this true but its not necessarily true
 - **Linearly independent:** Full rank system, no perfect multicollinearity between variables
 - **Errors are uncorrelated and homoscedastic:** $E[\epsilon_i \epsilon_j | X] = 0$ for $i \neq j$ and $E[\epsilon_i^2 | X] = \sigma^2$, meaning the error terms are not changing with x and the variance is constant throughout the data (there is no value of x that has a smaller error variance)
 - **Error terms are normally distributed:** $\epsilon|X \sim N(0, \sigma^2)$, used to impose the asymptotic properties to determine the standard errors and confidence intervals on the betas

Linear Regression

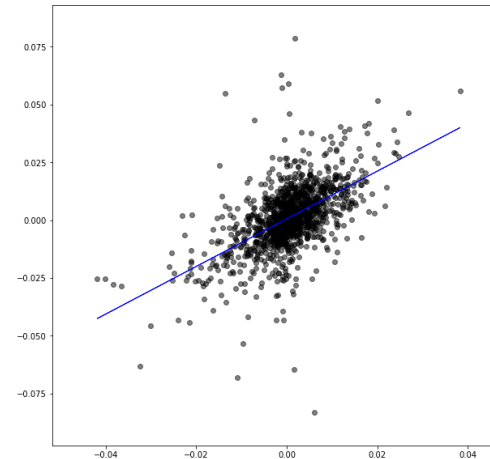
- The CAPM can be estimated using linear regression
 - $E(r_i) = r_f + \beta(E(r_m) - r_f)$
 - Rearrange to make it easier to estimate: $E(r_i) - r_f = \beta(E(r_m) - r_f)$
- Syntax: `sm.OLS(endo, exog)`
 - **Endo**: the dependent variable, y or left hand side
 - **Exog**: the independent variable(s), X or right hand side
- An intercept is not included in this model
 - To add one use the `sm.add_constant()` function

```
# Performing a simple/modified CAPM, not going to subtract rf
>>> capm = sm.OLS(data['aapl_rtns'], data['sp500_rtns'])
# y, X -> apple, sp500 (endogenous, exogenous)
>>> results = capm.fit()

>>> print(results.params)
[1.03475198]
# This means that Apple has a beta of 1.0347 to the market (sp500)
```

Linear Regression

- To plot the linear regression, generate predicted values for Y, which in this example are the returns for Apple
 - Calculating $\hat{y} = \beta x$, or more specifically $\widehat{r_{aapl}} = \beta r_{mrkt}$



```
>>> data['aapl_hat'] = results.predict(data['sp500_rtns'])
# Plot the scatter plot and the linear regression

>>> plt.subplots(figsize=(10, 10))

>>> plt.scatter(data['sp500_rtns'], data['aapl_rtns'], color='black', alpha=0.5)

>>> plt.plot(data['sp500_rtns'], data['aapl_hat'], color='blue', linewidth=1)

>>> plt.show()
```

Linear Regression

- When a linear regression is performed the coefficients are a point estimate; the confidence interval must be reviewed to learn if they are significant
 - The statsmodels package will automatically provide summary statistics, using the `.summary()` method for the results object

```
>>> print(results.summary())
OLS Regression Results
=====
Dep. Variable:          aapl_rtns      R-squared:                0.322
Model:                  OLS           Adj. R-squared:           0.322
Method:                 Least Squares  F-statistic:              598.1
Date:                   Thu, 02 May 2019  Prob (F-statistic):      2.28e-108
Time:                   17:47:34        Log-Likelihood:           -1979.4
No. Observations:       1258           AIC:                     3961.
Df Residuals:           1257           BIC:                     3966.
Df Model:                1
Covariance Type:        nonrobust
=====
               coef    std err          t      P>|t|      [0.025     0.975]
-----
sp500_rtns      1.0348      0.042     24.456      0.000      0.952     1.118
=====
Omnibus:                 209.489    Durbin-Watson:           1.933
Prob(Omnibus):            0.000    Jarque-Bera (JB):        3680.503
Skew:                     0.080    Prob(JB):                 0.00
Kurtosis:                 11.378    Cond. No.                 1.00
=====
```

Linear Regression

- Bootstrapping is a method to generate a distribution for coefficients using the data set (non-parametric or empirical)
 - The data set is permuted (with replacement) 'n' times. The total number of observations do not change, and the pairs are maintained
 - For each permutation iteration, the coefficients are estimated using OLS
 - This will generate 'n' estimates for the coefficients
 - Store all the coefficient estimates to create an empirical distribution
 - The intuition is that the data set is a representative sample of the population, so draws from the data set (samples) are equivalent to draws from the population

Dataset		Bootstrap (Iteration 1)		Bootstrap (Iteration 2)	
y	x	y	x	y	x
5	3	5	3	3	5
3	5	4	12	3	5
4	12	4	12	2	4
12	9	2	4	1	2
6	3	6	3	3	2
2	4	3	2	2	4
4	4	2	10	6	3
3	2	1	2	1	2
2	10	2	10	2	10
1	2	3	2	12	9

Linear Regression

- To automate this process, create a function that takes the data as x and y, along with the number of boot strap iterations
 - **X**: for CAPM will be the market returns
 - **Y**: for CAPM will be the stock returns
 - **Nboot**: note this should always end in 9 if the original estimate is added back in. In the example below nboot=999, and the original estimate is also calculated which will return a total of 1000 estimates for alpha and beta
 - Original estimate refers to the regression coefficient using the sample

```
>>> def bootcoef(X, y, nboot=999):
#Range of sample index, this allows for pairs sampling
...     inds = np.arange(len(X))
...     temp_reg = sm.OLS(y, X).fit()
...     bootresults = temp_reg.params.tolist()
...     for i in range(nboot):
...         # generate a random set of integers to select a bootstrap sample from data
...         bootint = np.random.choice(inds, len(inds))
...         Xboot = X[bootint]
...         yboot = y[bootint]
...         temp_reg = sm.OLS(yboot, Xboot).fit()
...         bootresults.extend(temp_reg.params.tolist())
...     return(bootresults)

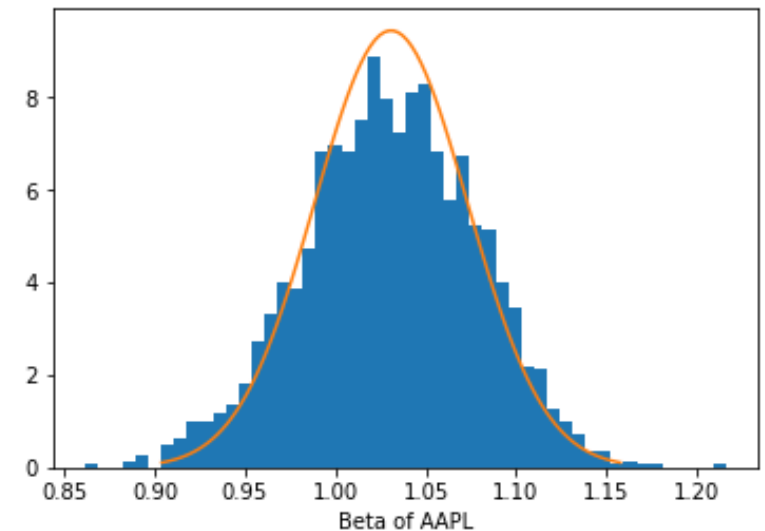
>>> np.random.seed(42)
>>> bootCAPM = bootcoef(data['sp500_rtns'].values, data['aapl_rtns'].values, nboot = 1999)
```

Linear Regression

- The boot strapped results can be plotted to see the distribution, along with overlaying the parametric distribution, which in this case is the normal distribution.
 - The mean and variance of the coefficient is required to plot the parametric distribution, in this case our beta
 - These values can be pulled from the model
 - .params returns a list of parameters
 - .cov_params returns a list of var/cov of the parameters
 - .conf_int() provides 95 percentile confidence interval

```
#Display the Histogram and Normal Distribution (parametric)
>>> beta_mean = results.params.values[0]
>>> beta_var = results.cov_params().values[0]
>>> beta_std = np.sqrt(beta_var)

>>> x_numbers = np.linspace(beta_mean - 3*beta_std, beta_mean + 3*beta_std, 100)
>>> plt.hist(bootCAPM, bins=51, density=True)
>>> plt.plot(x_numbers, norm.pdf(x_numbers, beta_mean, np.sqrt(beta_var)))
>>> plt.xlabel('Beta of AAPL')
>>> plt.show()
```



Time Series with *statsmodels*

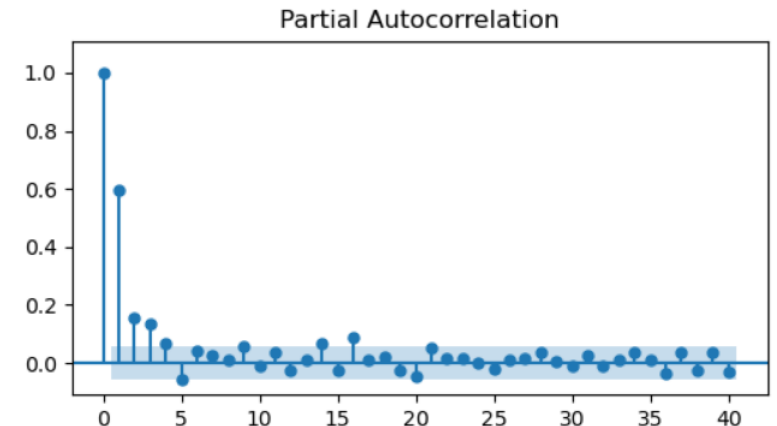
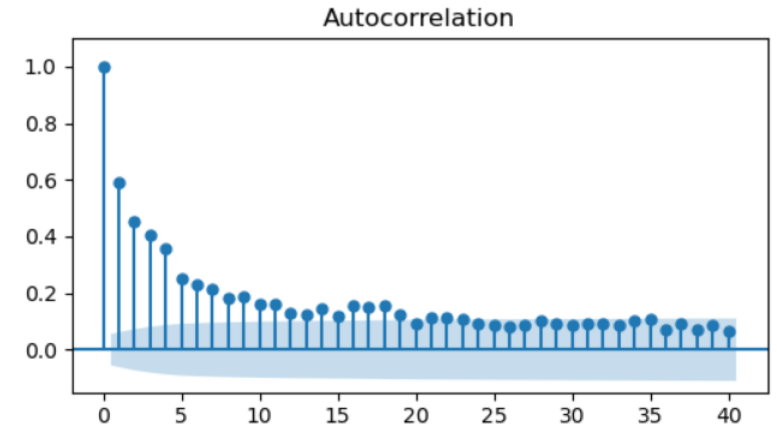
Time Series

- Time Series Analysis is used to find a relationship through time, and forecast (predict) the future based on historical data
- Time Series can be thought of as a linear regression on lagged variables
 - **AR**: auto regression, how many lags.
 - **I**: Number of differences ($y(t) - y(t+1)$)
 - **MA**: Moving average window
- Benefit of not using a packaged ARIMA model
 - Can control the specification better
 - Built in drift handling
 - Easier to construct hierarchical models
- General AR Model Specification:
 - $y_{t+1} = \delta + \phi y_t + \epsilon_t$
 - Long-run average: $\frac{\delta}{1-\phi}$
 - The model is considered stationary if $\phi < 1$, otherwise the process can 'blow-up'. Some more advanced modeling techniques can relax this condition for a few periods.

Time Series

- Typically, the ACF (autocorrelation function) and the PACF (partial autocorrelation function) are reviewed before modeling to get an idea of how many lags (AR and MA) to include in the model
- A simplified explanation is that
 - ACF uses all the data, it is useful to determine how many MA lags to include
 - PACF uses data incrementally, it is useful to determine how many AR lags to add

```
>>> fig = plt.figure(figsize=(10,10))
>>> ax1 = fig.add_subplot(2,1,1)
>>> fig = sm.graphics.tsa.plot_acf(
    ts500['vol'].values, lags=40, ax=ax1)
>>> ax2 = fig.add_subplot(2,1,2)
>>> fig = sm.graphics.tsa.plot_pacf(
    ts500['vol'].values, lags=40, ax=ax2)
>>> fig.show()
```



Time Series

- `.tsa.ARMA(data, (p,q)).fit(dis=False, trend='c')`
 - **Data**: the time series data being modelled
 - **(p,q)**: p is the number of AR lags, and q is the number of MA lags to include in the model
 - **Disp**: if True will display the convergence information
 - **Trend**: 'c' will include a constant (trend), 'nc' will remove the constant (no trend)
- Python will flag an error if the model is not stationary
 - It is best to code a try/except to handle the error if running a production script

```
>>> arma = sm.tsa.ARMA(ts500['vol'], (1,5)).fit(dis=False)
```

```
>>> print(arma.summary())
```

```
>>> arma = sm.tsa.ARMA(ts500['vol'], (1,22)).fit(dis=False)
```

```
ValueError: The computed initial AR coefficients are not stationary
```

```
You should induce stationarity, choose a different model order, or you can pass your own start_params.
```

Time Series

- `.forecast(steps=1, exog=None, alpha=0.05)`
 - **Steps**: number of out-of-sample steps to forecast
 - **Exog**: used to pass exogenous variables if the model is an ARMAX model, where the X is for additional non-lagged variables
 - **Alpha**: the level for which to produce the confidence intervals
- This function will return three results:
 - An array containing the forecast
 - An array containing the standard error (stderr) for the forecast
 - An array containing the confidence intervals for the forecast

```
>>> y_T, y_T_stderr, y_T_conf_int = arma.forecast(steps=5)

# The one step ahead forecast to see the structure
>>> arma.forecast(steps=1)
(array([0.17562662]), array([0.55567033]), array([[ -0.91346723,  1.26472046]]))
```

Time Series

- Plotting the data, estimation ($\hat{y}$) and forecast
 - Extract the fitted values ($\hat{y}$) from the model using `.fittedvalues`
 - The index passed from the DataFrame will be carried through
 - Create a forecast and combines the results into a DataFrame
 - Merge the forecast and confidence intervals into a DataFrame
 - The index will need to be extended before combining
 - Plot the data, model and forecast
 - Use the `.fill_between()` method to create the shaded confidence band on the plot

```
# Let's plot the 2018 data with the ARMA model using the built in functions
```

```
>>> y_hat = arma.fittedvalues
```

```
>>> y_T, y_T_stderr, y_T_conf_int = arma.forecast(steps=5)
```

```
>>> tsfore = pd.DataFrame(y_T, columns=['y_T']).join(  
    pd.DataFrame(y_T_conf_int,  
    columns=['y_T_low', 'y_T_upper']))
```

```
# Need to set index
```

```
>>> last_day = max(ts500.index)
```

```
>>> tsfore['date'] = [pd.to_datetime(last_day +  
    pd.offsets.BDay(i+1)) for i in range(5)]
```

```
>>> tsfore.set_index('date', inplace=True, drop=True)
```

```
# Now plot the data plus the forecast
```

```
>>> ts500.loc['2018:', 'vol'].plot()
```

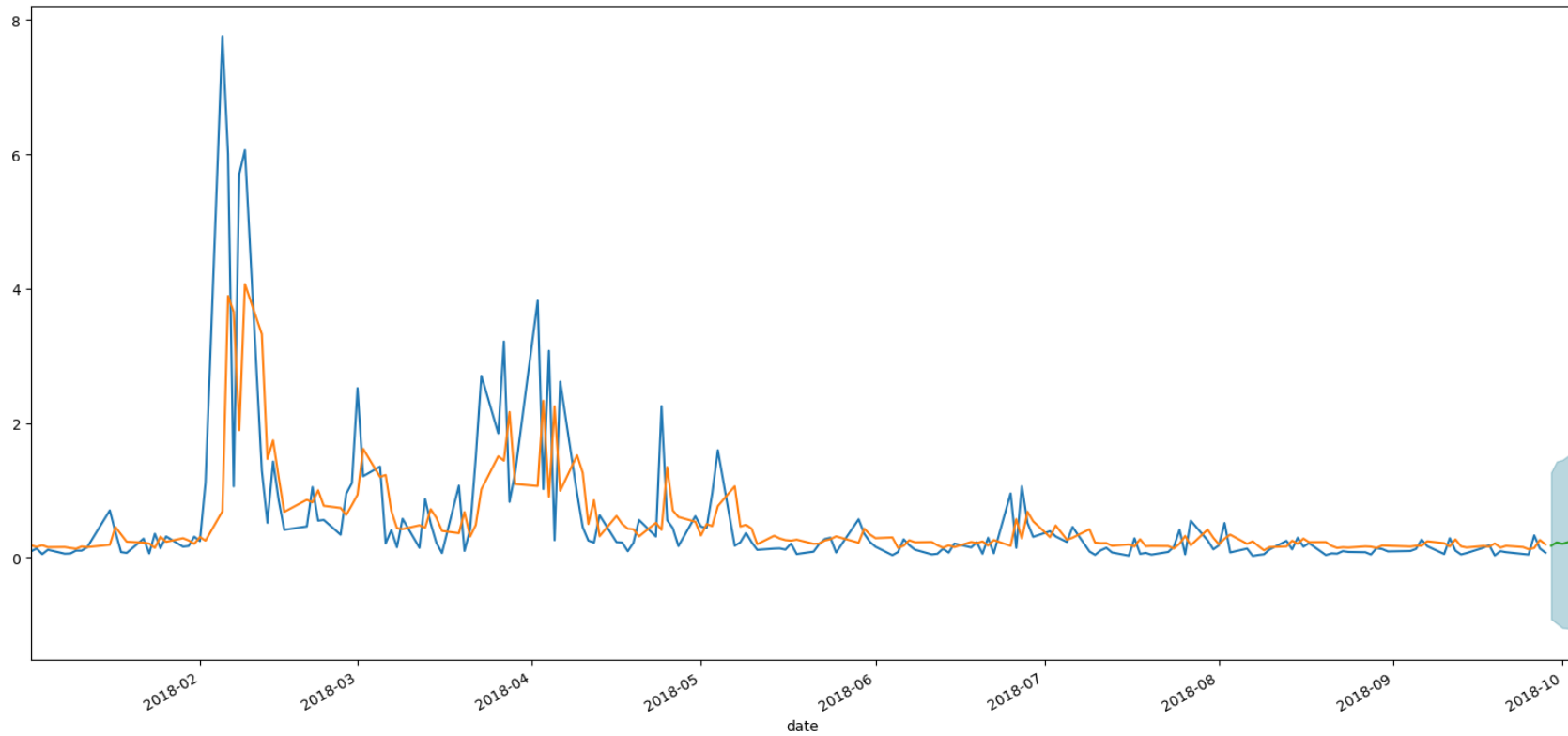
```
>>> y_hat.loc['2018:'].plot()
```

```
>>> tsfore['y_T'].plot()
```

```
>>> plt.fill_between(tsfore.index, tsfore['y_T_low'],  
    tsfore['y_T_upper'], color = '#539caf', alpha = 0.4)
```

```
>>> plt.show()
```

Time Series



Using *SciPy*

Optimization

Using SciPy – Optimization

- The SciPy library is a core python package providing numerical functions and is the parent package for Numpy
- Includes many common numerical functions, including Integration, Optimization, Interpolation and many others
- Many of these functions are commonly used in Finance applications, focus will be placed on the optimization function
 - Optimization is a common problem in finance, especially in portfolio selection
 - The minimization function will be used to find the weights of the optimal portfolio

```
>>> from scipy.optimize import minimize  
>>> from scipy.optimize import LinearConstraint
```

Portfolio Optimization

Portfolio Optimization – Overview

- Brief overview of portfolio theory
 - A collection of investment assets is called a portfolio. The weights of the portfolio are calculated by their respective values ($\sum w_i = 1$)
 - For example, consider two assets with \$3000 in asset 1 and \$2000 in asset 2. The weight of asset 1 is 0.6 and the weight of asset 2 is 0.4
 - The expected return of the portfolio is the weighted average of the expected return of the individual assets
 - For example, if asset 1 has an expected return of 5% and asset 2 has an expected return of 10% the portfolio expected return would be 7% using the weights above
 - Why form a portfolio? If asset 2 is expected to make more than asset 1, why not invest fully in asset 2?
 - The purpose is to diversify risk, think of the phrase “don’t put all your eggs in one basket”
 - To illustrate this, consider asset 1 has a standard deviation 10% and asset 2 has a standard deviation of 20%, with a correlation of 0.3 between the two, the portfolio variance is
$$\sigma_p^2 = w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2w_1 w_2 \rho_{1,2} \sigma_1 \sigma_2$$
$$\sigma_p^2 = 1.288\%$$
$$\sigma_p = 11.349\%$$
 - The investor is able to increase their portfolio return by 2% over just investing in asset 1 while only adding 1.4% to the risk of the portfolio

Portfolio Optimization – Overview

- The classical portfolio selection problem is to minimize risk (variance/standard deviation) while maximizing return
 - One constraint occurs by formulation of the problem, the desired return
 - The simplest constraint is that the portfolio weights have to sum to one not to break equilibrium rules
 - Other constraints can be added, such as no short selling ($w \geq 0$), or not have more than x% in a security ($w \leq 0.05$)
- Our problem is as follows
 - $\min_w w^T V w$ s.t. $1 = \sum w_i$ and $E[r] = \sum (w_i r_i)$
- The objective function, portfolio variance, is the same as in the example but now in matrix notation.
 - Remember that $\rho_{1,2} \sigma_1 \sigma_2 = \sigma_{1,2}$ where $\sigma_{1,2}$ is the covariance between assets 1 and 2

Portfolio Optimization – Minimize

- `Minimize(func, x0=None, args=(), constraints=(), bounds=())`
 - **func**: the objective function, which should be minimized. Can be a defined function or a lambda function
 - **x0**: the starting values for x. For complicated functions the starting conditions can have an effect on the result
 - **args**: passed as a tuple to the func
 - **constraints**: a tuple of the defined constraints for the problem
 - **bounds**: a sequence or tuple of (min, max) tuple pairs for each element in x

```
#Function returns the variance of the portfolio given weights and covariance matrix
>>> def optw(w, *args):
...     return(np.matmul(np.matmul(w,V),w))

>>> weight = minimize(optw, w, (V),
                      constraints=(constraint_1,constraint_2),
                      options={'maxiter':1000})
```

Portfolio Optimization – Constraints

- `LinearConstraint(A, lb, ub)`
 - **A**: is a matrix/array which has the coefficients for the constraint equation
 - Remember that no coefficient is equivalent to having a 1
 - **Lb**: the lower limit of the constraint
 - **Ub**: the upper limit of the constraint
 - Set Lb equal to Ub for equality constraint
- The constraints are specified in the form $lb \leq A \cdot X \leq ub$
- In older versions of SciPy, the constraints need to be set up as a dictionary

```
# constraint for the weights
>>> constraint_1 = LinearConstraint(A, 1, 1)

# for the return constraint, the asset returns are the coefficients
>>> constraint_2 = LinearConstraint(mu, expect_return, np.inf)

# for older versions of scipy use:
>>> constraint_1 = {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}
>>> constraint_2 = {'type': 'ineq', 'fun': lambda x: np.sum(x * mu ) - expect_return}
```

Portfolio Optimization – Constraint 1

- How to go from $1 = \sum w_i$ to python code
 - The constraints are specified in the form $lb \leq A \cdot x \leq ub$
 - The dot operator takes two matrices, performs element wise multiplication and then sums them. In notation,
$$A \cdot X = \sum a_i x_i = a_1 x_1 + a_2 x_2 + \dots + a_n x_n$$
 - Lining up the two on top of each other
$$\begin{array}{rcl} 1 & = & \sum w_i \\ A \cdot X & = & \sum a_i x_i \end{array}$$
 - We can see that $A \cdot X = 1$, X is the matrix of unknown weights.
 - If a variable, such as w_i , does not have a written coefficient in front of it, it actually has a 1. This means that in this case the A matrix is comprised of all ones
 - Since the weights must add up to 1, the lower and upper bounds are specified to be 1, resulting in $1 \leq 1 \cdot w \leq 1$

```
# constraint for the weights, n = # of stocks to be analyzed
>>> A = np.matrix([1]*n)
>>> constraint_1 = LinearConstraint(A, 1, 1)
```

Portfolio Optimization – Constraint 2

- How to go from $E[r] = \sum(w_i r_i)$ to code
 - The constraints are specified in the form $lb \leq A \cdot x \leq ub$
 - Following the same logic as above, stacking the formulas
$$E[r] = r^T w = \sum(r_i w_i)$$
$$A \cdot X = \sum(a_i x_i)$$
 - The difference now is that there is a coefficient in front of the unknown weight variable, the return of the asset. This means that the A matrix is the matrix of expected asset returns.
 - The left hand side, $A \cdot X$, is the specified return the investor wishes to achieve on the portfolio. This is considered to be the lower bound, the minimum the investor wishes to achieve. The upper bound is left unconstrained, infinity, because if a higher return of the same variance is possible it should be attained.
 - This gives $E[r] \leq r \cdot w \leq \infty$

```
# series of average returns of each stock (expected returns)
>>> mu = np.mean(rtns, axis=0) * 100
# constraint for returns
>>> constraint_2 = LinearConstraint(mu, expect_return, np.inf)
```

Portfolio Optimization – Constraints as Dictionaries

- In previous versions of SciPy (packages with versions <1.0), the LinearConstraint function did not exist
- Instead, each constraint is set up as a dictionary with the following keys and values:
 - **type**: 'eq' for equality and 'ineq' for inequality
 - Equality constraint means that the constraint function result needs to equate to zero, while inequality means the function result needs to be non-negative
 - **fun**: the function defining the constraint
 - Typically set up as a lambda function
 - The inputs to the function are the values of x in the optimization's minimize function
- Specifying the constraints in the older format:
 - Constraint 1: $1 = \sum w_i$ needs to be re-arranged as an equality of 0, as $\sum w_i - 1 = 0$
 - Constraint 2: $E[r] = \sum (w_i r_i)$ needs to be re-arranged as an inequality, $\sum (w_i r_i) - E[r] \geq 0$

```
# series of average returns of each stock (expected returns)
>>> mu = np.mean(rtns, axis=0) * 100
>>> constraint_1 = {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}
>>> constraint_2 = {'type': 'ineq', 'fun': lambda x: np.sum(x * mu ) - expect_return}
```

Portfolio Optimization – Bounds

- bounds = tuple of (min, max) pairs
- The bounds argument of the minimize function provides minimum and maximum limits for the x variables being sensitized in the optimization
- For a portfolio optimization, bounds of (0,1) are usually provided to restrict for short selling and ensure none of the asset weightings exceeds 100%
 - The max is also sometimes restricted to a maximum exposure in the portfolio (e.g. max 25% weighting)

```
# bounds for the weights to restrict short-selling
>>> n = mu.shape[0] #number of assets in portfolio
>>> bound = (0.0,1.0) # (min, max) boundaries for any one of the assets
>>> bounds = tuple(bound for asset in range(n)) #tuple for all assets

# updating the minimize function:
>>> weight = minimize(optw, w, (V),
                      constraints=(constraint_1,constraint_2),
                      bounds=bounds,
                      options={'maxiter':1000})
```

Portfolio Optimization – Results

- The results are stored in a result object
 - **.x**: stores the solution vector in a NumPy array
 - **.success**: True/False if the solver was able to find a solution
 - **.nit**: The number of iterations required to find a solution
 - **.message**: The cause of termination

```
>>> weight
fun: 0.18184203971590196
jac: array([-0.10255267, -0.08567294, -0.05414635, -0.06006911, -0.09526219,
           0.03190438,  0.05650454,  0.02412239, -0.09328883,  0.02637638,
           0.15002657,  0.11958694,  0.13675068,  0.17859602,  0.31597819,
           0.26219905,  0.27370915,  0.27453952,  0.37605105,  0.22642007,
           0.36682575,  0.37303495,  0.31686172,  0.47696703,  0.51525565])
message: 'Optimization terminated successfully.'
nfev: 781
nit: 28
njev: 28
status: 0
success: True
x: array([-0.11165125,  0.00122382, -0.01348225,  0.00306191, -0.03130665,
          0.06896753, -0.01649814, -0.0042416 ,  0.0132445 ,  0.03334851,
          -0.13659692,  0.04308279,  0.00444776,  0.04963469,  0.00505731,
           0.32991776,  0.15612027,  0.03729035, -0.00815732,  0.04175576,
           0.32737215,  0.17916443, -0.02932554,  0.02308353,  0.03448659])
```

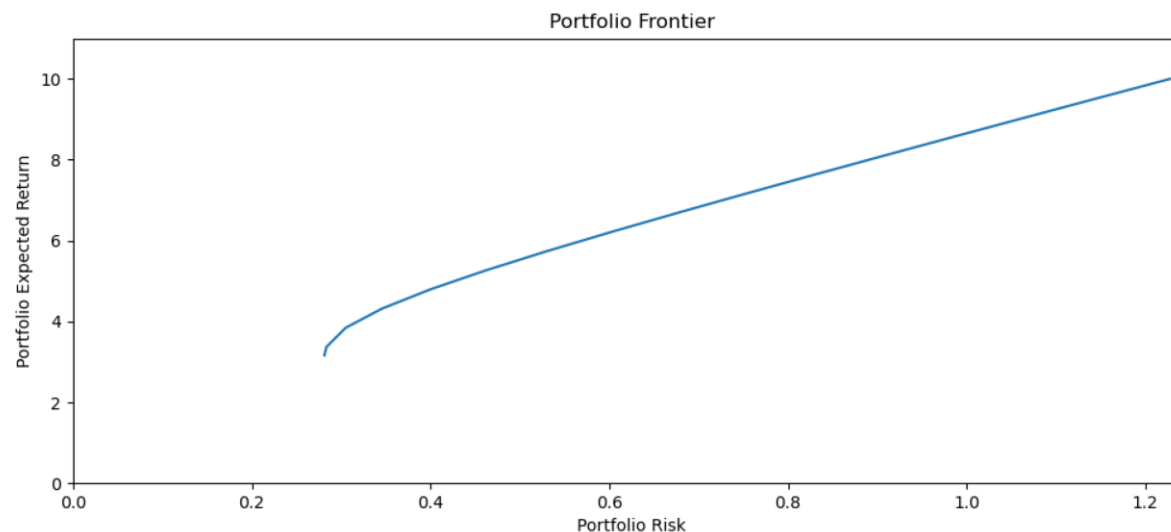
Portfolio Optimization – Frontier

- The portfolio frontier can be plotted (asset or efficient frontier if talking about the market)
 - Loop through a range of returns to calculate then find the minimum portfolio variances. Some returns are not achievable, so use the .success attribute to filter
 - Find the portfolio expected return and variances
 - Plot the return versus standard deviation (risk)

```
>>> weights = []

>>> for port_return in np.linspace(1,10, 20):
...     constraint_2 = LinearConstraint(mu, port_return, np.inf)
...     weight = minimize(optw, w, (V),
...                       constraints=(constraint_1,constraint_2),
...                       options={'maxiter':1000})
...     if weight.success == True:
...         weights.append(weight.x)
```

Portfolio Optimization – Frontier



```
# Find the portfolio returns and variances
```

```
>>> portmu = []
```

```
>>> portvar = []
```

```
>>> for w in weights:
```

```
    portmu.append(w @ mu)
```

```
    portvar.append(w @ V @ w)
```

```
>>> portstd = np.sqrt(portvar)
```

```
>>> plt.plot(portstd, portmu)
```

```
>>> plt.title('Portfolio Frontier')
```

```
>>> plt.xlim([0,1.25])
```

```
>>> plt.ylim([0,11])
```

```
>>> plt.xlabel('Portfolio Risk')
```

```
>>> plt.ylabel('Portfolio Expected Return')
```

```
>>> plt.show()
```

Using *scikit-learn* (*sklearn*)

Using *scikit-learn* (*sklearn*)

- Scikit-learn (sklearn) is a large data analysis packaged used for applying machine learning algorithms
 - Typically, only the necessary modules are imported in order to save memory and time
 - The package contains functions for
 - Classification, regression, cluster detection, dimensionality reduction, data preprocessing and model selection.
- Focus will be on K-Means Clustering to group data sets into clusters and Principal Component Analysis to perform factor analysis on financial data

```
>>> from sklearn.cluster import Kmeans  
>>> from sklearn.decomposition import PCA
```

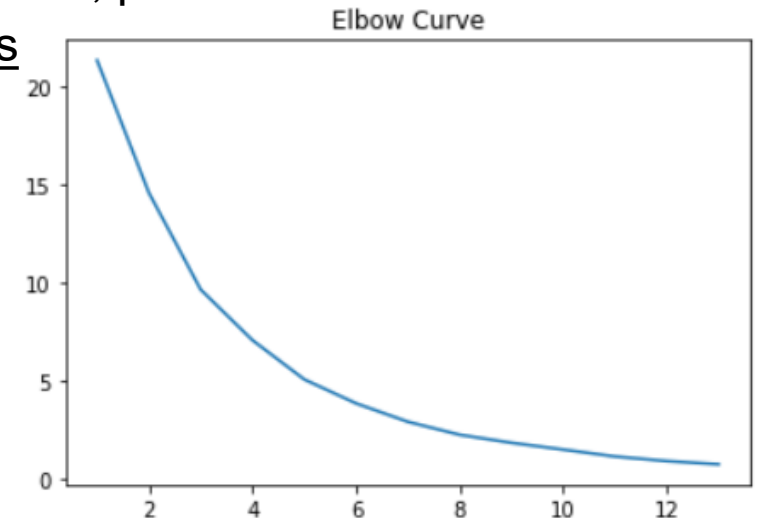
K-Means Clustering

Machine Learning Algorithms

- Python has become very popular in the data science community due to the large amount of Machine Learning and AI algorithms available through third party packages
- **Scikit-learn** is the most commonly used package for Machine Learning and has algorithms for the following applications:
 - **Classification**: identifying which category an object belongs to
 - E.g. after training a model what is spam and what is not, the classifier model will “classify” new emails
 - **Regression**: predicting continuous-valued attributes associated with independent variables
 - E.g. predicting returns of portfolio based on certain factors (market risk premium, size premium, etc.)
 - **Clustering**: automatic grouping of similar objects into sets
 - E.g. allocating customers into different categories based on spending habits and other characteristics
- Clustering is a type of Unsupervised Machine Learning algorithm
 - Unsupervised Learning allows us to approach problems with little or no idea what our results should look like. We can derive structure from data where we don't necessarily know the effect of the variables
 - We can derive this structure by clustering the data based on relationships among the variables in the data
 - Example: provide a data set of thousands of trading customers and find a way to automatically group the clients into groups that are similar or related by different variables, such as the # of trades, commissions generated, \$ volume traded, etc.

K-Means Clustering

- There are many clustering algorithms available in the *sklearn* package and a good overview and comparison can be found at:
 - <https://scikit-learn.org/stable/modules/clustering.html>
- The K-Means algorithm is one of the most common cluster algorithms
 - The algorithm clusters data by trying to separate samples in “n” groups of equal variance, minimizing a criterion known as the inertia or within-cluster sum-of-squares.
 - When using the algorithm in python, the number of clusters needs to be specified.
 - For further reading on how the algorithm categorizes data into cluster, please see:
 - <https://scikit-learn.org/stable/modules/clustering.html#k-means>
- One of the major assumptions when using K-Means is picking the optimal number of clusters
 - The “Elbow Method” is one method of finding the number of clusters and consists of looping through different scenarios and looking at the SSE (Sum of Squared Errors) then plotting the SSE's on a graph for each # of clusters picked



K-Means with sklearn

- Documentation from version 0.20:
 - <https://scikit-learn.org/0.20/modules/generated/sklearn.cluster.KMeans.html#sklearn.cluster.KMeans>
- `KMeans(n_clusters=8)`
 - If `n_clusters` is not set, the default is 8 clusters
- `KMeans().fit(X)`
 - Returns a fitted model object
 - `X`: data to be decomposed
- The result object has several attributes that can be accessed
 - `.cluster_centers_`
 - This provides an array of the x, y coordinates of the centroids of each cluster found
 - `.labels_`
 - An array of the cluster number for each data point that was analyzed
 - `.inertia_`
 - Sum of squared distances of samples to their closest cluster center

```
>>> X = mu_var_rtns.values #Converting std and returns into numpy array for the model
>>> kmeans = KMeans(n_clusters = 5).fit(X)
>>> centroids = kmeans.cluster_centers_
>>> princomp = PCA().fit(simrtns)
```

Cluster of Securities Based on Risk/Return Profile

- In this example, securities are grouped into clusters based on their expected returns and variance
- Given a DataFrame of daily returns of multiple securities called *rtns*, the following code creates the X variable using the mean returns and standard deviations of each security

	AAPL	AXP	BA	CAT	CSCO	CVX	DIS	DWDP	GE	GS	...	NKE
Date												
2013-11-30	0.070053	0.048900	0.032485	0.014875	-0.058067	0.029244	0.028430	-0.010388	0.019893	0.053679	...	0.044614
2013-12-31	0.008902	0.057459	0.016685	0.073404	0.055530	0.020173	0.096445	0.144879	0.059895	0.049248	...	-0.000072
2014-01-31	-0.107697	-0.060516	-0.082277	0.047728	-0.015723	-0.106317	-0.049607	0.025000	-0.103461	-0.074128	...	-0.073627
2014-02-28	0.057511	0.073630	0.035024	0.032584	-0.005021	0.042322	0.112932	0.070314	0.022385	0.017620	...	0.081430
2014-03-31	0.019953	-0.013694	-0.026606	0.024750	0.028441	0.031041	-0.009157	0.005060	0.016490	-0.015620	...	-0.056705

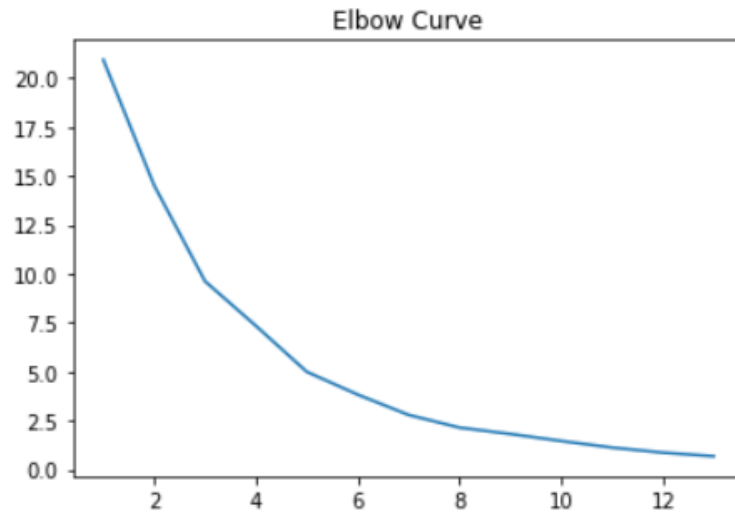
	risk	return
AAPL	6.787209	2.276223
AXP	5.366346	0.718297
BA	6.684699	2.238272
CAT	6.832630	1.546707
CSCO	5.533057	1.723833
CVX	5.524930	0.518233
DIS	5.205335	1.170476
DWDP	6.092664	1.270462
GE	5.608769	-0.950579
GS	6.197503	0.862902

```
# create five factor portfolios
mu_rtms = rtms.mean() * 100
var_rtms = rtms.var() * 100
std_rtms = rtms.std() * 100

#Storing the Monthly Average Returns and STD in a new dataframe
mu_var_rtms = pd.concat([std_rtms, mu_rtms], axis = 1)
mu_var_rtms.columns = ['risk','return']
mu_var_rtms
```

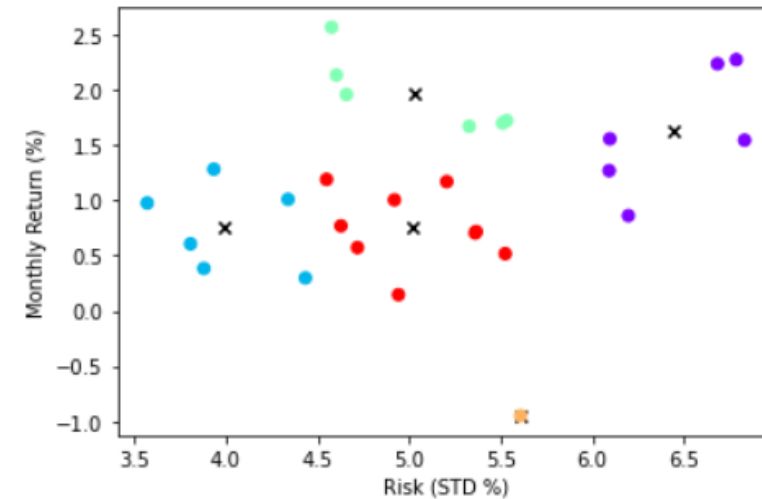
```
# Running the k-means model
X = mu_var_rtms.values
kmeans = KMeans(n_clusters = 5).fit(X)
centroids = kmeans.cluster_centers_
categories = kmeans.labels_
mu_var_rtms['Cluster'] = kmeans.labels_
```

K-Means Cluster Visualizations



```
sse = []
for k in range(2,15): #number of clusters
    kmeans = KMeans(n_clusters = k)
    kmeans.fit(X)
    sse.append(kmeans.inertia_)

#Quick plot of Elbow Curve
plt.plot(range(1,len(sse)+1), sse)
plt.title("Elbow Curve")
plt.show()
```

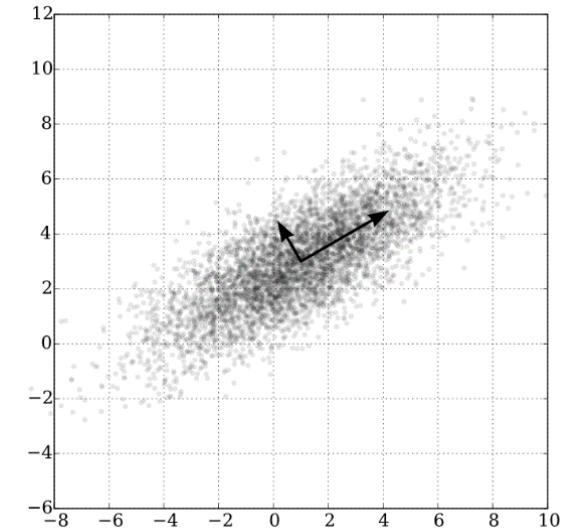


```
kmeans = KMeans(n_clusters = 5).fit(X)
centroids = kmeans.cluster_centers_
plt.scatter(centroids[:,0], centroids[:,1], c = 'black', marker='x')
plt.scatter(X[:,0],X[:,1],
            c = kmeans.labels_,
            cmap = "rainbow")
plt.ylabel("Monthly Return (%)")
plt.xlabel("Risk (STD %)")
plt.show()
```

Principal Component Analysis

Principal Component Analysis

- Principal Component Analysis (PCA)
 - PCA is a statistical method used to “de-couple” a system of correlated variables
 - Creates a set of axis all orthogonal to each other, but in the direction of greatest variance in the data
 - The dimensionality of the system can be reduced by only selecting a subset of the components
- Relation to finance
 - PCA can be used to create “factor” portfolios from a set of assets, or other economic variables that are correlated. The “factors” are linear combinations of the observable variables.
 - Do not confuse this method with Factor Analysis, which treats the factor as a latent (unobservable) variable to be estimated.
 - Used in fixed income and derivative models where there are several observable variables, but the relationship and interaction is complex



Principal Component Analysis.

- Once the Principal Components are found, linear regression (OLS) can be used to find the sensitivity of asset to the factors
 - The OLS equation will be in the form $r_i = l_1F_1 + l_2F_2 + \dots + l_nF_n$
 - Where r is the asset returns, F are the factor portfolios, and l is the sensitivity (coefficients)
- The first step is to find the factor loadings
 - This is done through PCA, where the eigenvector is equivalent to the weights for that asset in the factor portfolio
- Once the loadings are found, the next step is to rescale them so they add to one
 - In finance, when using a market/equilibrium model (tradable securities) factor portfolios must sum to one
 - The factor is represented by a portfolio that is long some assets and short others
 - This is similar thinking to the Fama-French factor portfolios: Small minus Big, High minus Low, etc.
 - If using non-tradeable assets, such as GDP or inflation, this step is not required but this also changes the interpretation of the coefficients in the regression
- Using the scaled weights create the factor portfolios by multiplying the respective weights with the assets

Principal Component Analysis

- `PCA(n_components=None)`
 - If `n_components` is not set, the number of components that are kept is `min(n_samples, n_features)`
- `PCA().fit(X)`
 - Returns a fitted model object
 - `X`: data to be decomposed
- The result object has several attributes that can be accessed
 - `.explained_variance_ratio_`
 - This provides how much each component explains the variance of the system
 - To perform the analysis all components or a subset of components can be used
 - `.components_`
 - Returns the eigenvectors of the dataset

```
>>> pca = PCA(n_components)
>>> pca.fit(X)
>>> princomp = PCA().fit(simrtns)
>>> princomp.explained_variance_ratio_
array([0.1588896 , 0.13428155, 0.11755048, 0.08050946, 0.07869987, 0.06905565, 0.05377585,
        0.04607226, 0.04394373, 0.03312868, 0.03214288, 0.0292903 , 0.02522983, 0.02379697,
        0.0186408 , 0.01268088, 0.01215141, 0.00827529, 0.00743229, 0.00458515, 0.00306756,
        0.00284917, 0.00193188, 0.00124312, 0.00077535])
>>> sum(princomp.explained_variance_ratio_[:5])
0.569930962563556
```

Principal Component Analysis

- In this example, tradable assets are used to construct the factor portfolios. This means that market clearing conditions must hold, $\sum w_i = 1$.
- Because of the equilibrium constraint `.transform()` method cannot be used for this model, instead some manual steps are required
 - Scale the factor loadings to sum to one. After the scaling, the loadings can be interpreted as weights.
 - Matrix multiply the weights with the asset returns to find the factor portfolios

```
# create five factor portfolios
>>> comp = princomp.components_.T
>>> scale = np.sum(comp, axis=0)
>>> import numpy.matlib
>>> scales = np.matlib.repmat(scale, 25, 1)
>>> scaled_comp = comp/scales
>>> factors = np.matmul(simrtns ,scaled_comp[ : ,0:5])

# Now we can use these factor portfolios in a linear regression
```